

Mathitude — Payment Compliance

Ari · 2026-05-24

May 24, 2026

CONTENTS

Mathitude — Payment data handling & PCI posture	
How card collection actually works	
What Mathitude stores	
What Stripe stores	
Network path of a card number (one round trip)	
Restricted Stripe key usage	
Webhook signature verification	
Logging hygiene	
Quick mental model for Paula	
Sources	

Mathitude — Payment data handling & PCI posture

Date: 2026-05-24 **Audience:** Paula + anyone asking how cards are stored **TL;DR:** Mathitude never sees, never transmits, and never stores credit card numbers. All card data goes from the customer's browser directly to Stripe over an encrypted channel inside a Stripe-hosted iframe. Mathitude only ever holds Stripe's tokenized references (`pm_...` , `cus_...`) and last-4/brand for display.

How card collection actually works

1. The parent (or Paula on a parent's behalf, on `/admin/families/[id]`) sees a card form. The "card details" input is **not Mathitude's HTML**. It is a Stripe-hosted iframe rendered by `@stripe/react-stripe-js`'s `<CardElement>` component (`src/components/stripe/save-card-form.tsx`).
2. When the user types their card number, expiry, CVC, and ZIP, those keystrokes go into Stripe's iframe DOM. Our JavaScript cannot read them — the iframe is cross-origin, isolated by the browser.
3. On submit, our code calls `stripe.confirmCardSetup(clientSecret, ...)`. Stripe's JS reads the card values from its own iframe, POSTs them directly to `api.stripe.com` over TLS 1.2+, and returns a `PaymentMethod` token (`pm_...`) to our browser code.
4. Our code then calls our own server (`POST /api/stripe/payment-methods/finalize-new-card`) with only the `paymentMethodId` and `parentId` . **Card number never reaches our server.**
5. Our server attaches the `paymentMethod` to the parent's Stripe `Customer` (`cus_...`) via the Stripe Node SDK using our restricted secret key. The `Customer.invoice_settings.default_payment_method` pointer is updated when the user clicks "Set as default" + Save Changes.

This is Stripe's standard "Elements + SetupIntent" pattern. It is the flow Stripe recommends for SAQ A PCI eligibility — the lowest-burden PCI Self-Assessment Questionnaire because the merchant never handles cardholder data.

What Mathitude stores

In DynamoDB (`mathitude-staging-*` tables, AWS region `us-west-2` , all tables encrypted at rest via AWS-managed KMS keys):

Table	What we store	What we never store
<code>parents</code>	Stripe customer reference <code>stripeCustomerId</code> (e.g. <code>cus_R8xQ...</code>)	Card number, expiry, CVC
<code>payments</code>	Stripe <code>paymentIntent</code> + <code>charge</code> ids, amount in cents, status, brand+last4 (display only)	Card number, expiry, CVC, ZIP
<code>notifications</code>	"Brand + last4" for activity log (e.g. "Visa ending in 4242")	Card number, expiry, CVC

The four-digit last-4 + card brand are explicitly defined by PCI DSS as **non-sensitive** (they are insufficient to charge a card on their own). Stripe returns them to us alongside the `PaymentMethod` token; we display them so Paula can identify which card a customer is using ("Visa ending in 7710").

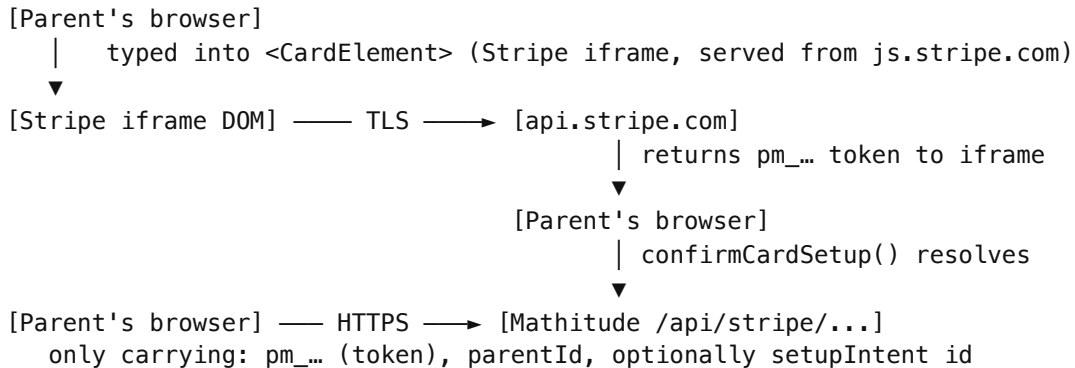
What Stripe stores

Stripe holds:

- Full PAN (card number)
- Card expiry
- CVV/CVC (only for the initial card-verification charge — Stripe never stores CVC long-term, per PCI rules)
- Cardholder name + billing ZIP
- Linkage between `customer` → `paymentMethod`

Stripe is **PCI DSS Level 1 compliant** (the highest tier). Their attestation of compliance is downloadable from their dashboard under Compliance → Documents. SOC 1 / SOC 2 reports are also available.

Network path of a card number (one round trip)



Mathitude's servers never appear on the card-number path. There is no configuration setting Paula or any future operator can change to make card numbers traverse our infrastructure — the Stripe iframe enforces this at the browser level.

Restricted Stripe key usage

Our backend uses a Stripe **restricted key** (`rk_...`) where possible (see `src/lib/server/stripe.ts:153`). The key is scoped to the minimum permissions our flows require:

- `paymentmethods:write` (attach/detach to customers)
- `customers:write` (create on first card save, update default)
- `paymentintents:write` (Phase 3 charge approval queue)
- `setupintents:write` (the save-card flow)

The key lives in DynamoDB (`mathitude-staging-secrets`), encrypted at rest, never echoed in any response — `getStripeMeta()` only returns `last4` of the key for diagnostic display in `/admin/settings/stripe`. An env-var fallback (`STRIPE_SECRET_KEY`) exists for local development.

Webhook signature verification

`POST /api/stripe/webhook` (`src/app/api/stripe/webhook/route.ts`) verifies every incoming Stripe webhook via the `Stripe-Signature` header against a webhook secret. Unsigned or stale-timestamp webhooks are rejected with 400. This prevents an attacker from sending forged

`payment_intent.succeeded` events to mark debts as paid.

Logging hygiene

No log statements in the codebase reference card numbers (`grep -r "cardNumber\|card_number\|pan" src/` returns zero hits as of this date). Stripe error objects sometimes include the last-4 of a declined card; those are safe to log.

Quick mental model for Paula

If your bookkeeper asked “is Mathitude storing credit cards somewhere?” — the honest answer is **no**. The cards live in Stripe’s vault. We have a key-card to the vault (the Stripe API key) and a list of who has which card (the `paymentMethod` and `customer` ids). We do not have the cards themselves. We could not give cardholder data to a hacker, a subpoena, or anyone else if asked, because we do not possess it.

For deeper compliance (SOC 2 Type 2 attestation by Mathitude itself), see `PAULA_FAQ.md` section 5. Path is ~3 months + a vendor like Vanta or Drata. Stripe + Clerk + AWS already provide their own SOC 2 attestations which cover the sub-processor portion of any Mathitude audit.

Sources

- Stripe Elements documentation: https://stripe.com/docs/payments/elementsSMART-PANTS_RESERVED_206SMARTPANTS_RESERVED_207
- Stripe PCI compliance overview: https://stripe.com/docs/security/stripeSMARTPANTS_P-RESERVED_210SMARTPANTS_RESERVED_211
- PCI SSC scope guidance for tokenization (cards never on merchant systems): PCI DSS v4.0 §3.3 + §3.5 (tokenization scope reduction)
- Repo files cited above:
 - `src/components/stripe/save-card-form.tsx`
 - `src/app/api/stripe/create-setup-intent/route.ts`
 - `src/app/api/stripe/payment-methods/finalize-new-card/route.ts`
 - `src/app/api/stripe/webhook/route.ts`
 - `src/lib/server/stripe.ts`
 - `src/lib/server/secrets.ts`

